GPU Acceleration of a Diabetic Retinopathy Classification Pipeline

Engineering Project — 2 Year | GPU Programming and Performance Optimization | 4 Weeks

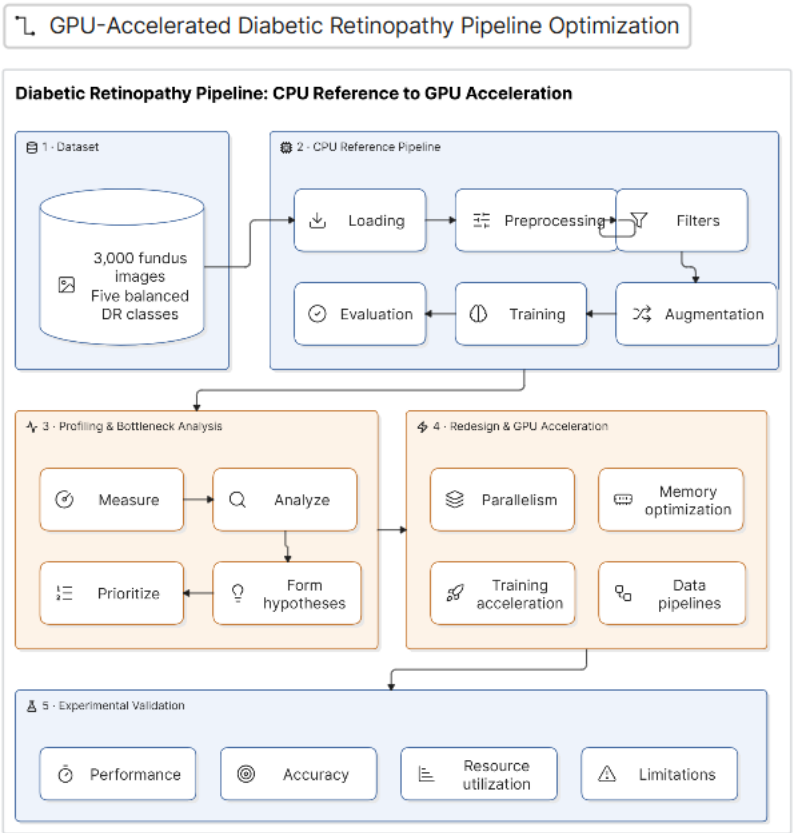
1. Project Overview

The objective of this project is to optimize an existing computer vision and deep learning pipeline for diabetic retinopathy (DR) classification using GPU acceleration.

Students will receive a functional CPU-based reference implementation and a dataset of 3,000 balanced retinal fundus images distributed across five diabetic retinopathy severity classes. They must first understand and profile the reference implementation, identify its computational bottlenecks, and then redesign and optimize selected components to improve execution time while preserving or improving classification performance.

The project **is not simply about moving code to a GPU**. It is about **using profiling evidence to guide architectural redesign, selecting appropriate optimization strategies, validating their correctness, and critically evaluating the results**.

Core engineering challenge



2. Learning objectives

By the end of the project, students should be able to:

1. Understand and execute an existing CPU-based image classification pipeline.
2. Profile Python applications and identify computational and memory bottlenecks.
3. Explain when GPU acceleration is beneficial and when it introduces overhead.
4. Redesign algorithms and data pipelines to exploit parallel computation.
5. Implement GPU acceleration using suitable high-level frameworks, such as PyTorch, CuPy, or Numba.
6. Optimize data transfer, memory allocation, batching, and CPU–GPU concurrency.
7. Accelerate CNN training using GPU execution, mixed precision, and appropriate data-loading strategies.
8. Design reproducible experiments and interpret performance measurements.
9. Evaluate the relationship between computational optimization and classification accuracy.
10. Communicate engineering decisions, results, trade-offs, and limitations in a technical report and oral defense.

3. Dataset and experimental setup

The instructor will provide the dataset and CPU reference implementation.

Parameter	Specification
Images	3,000 retinal fundus images
Classification	Five DR severity classes
Distribution	Balanced across the five classes
Image format	RGB; source resolution documented by the instructor
Reference implementation	Python, NumPy, OpenCV and PyTorch
Target hardware	CPU and NVIDIA GPU, where available
Duration	Seven weeks
Team size	2–3 students recommended

The dataset must be divided into training, validation, and test sets before any model development. If multiple images belong to the same patient, all images from that patient must remain in the same split.

The test set must not be used to select hyperparameters or optimize the model.

4. CPU reference implementation provided by the instructor

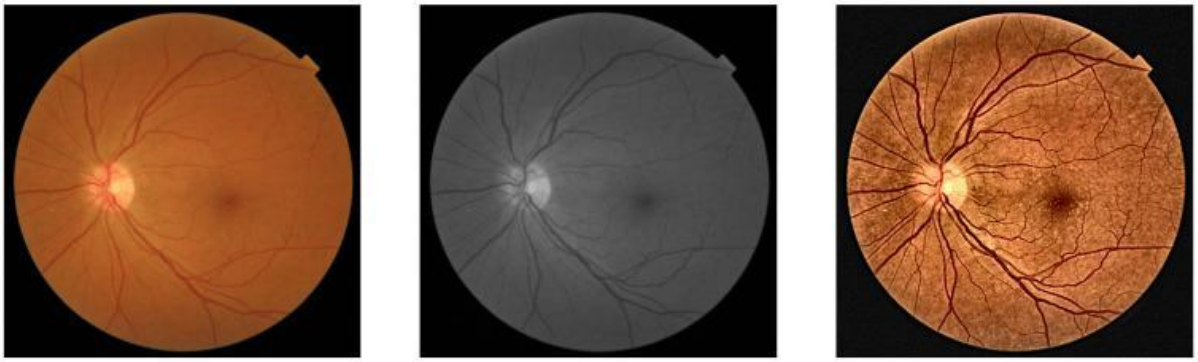
Students will receive runnable, documented source code containing the following modules.



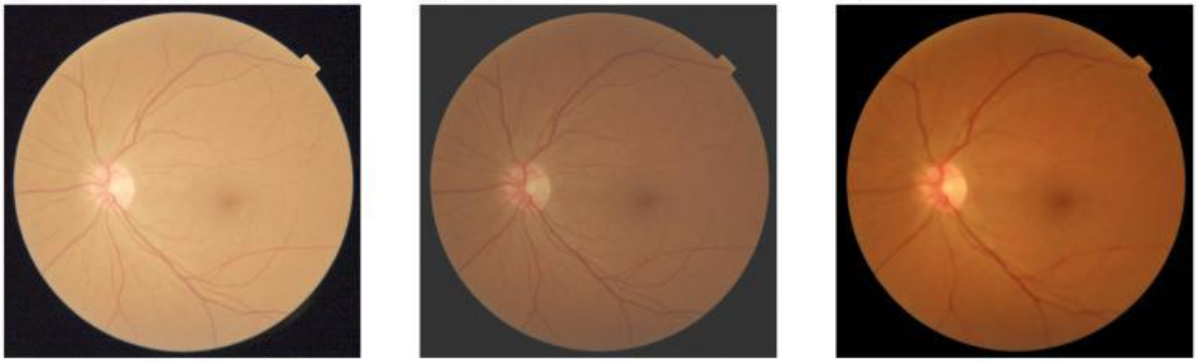
Module A — Image loading

Read images from disk, decode them, resize them to the target resolution, and prepare batches.

Investigate disk I/O, image decoding, repeated loading, and data-loading overhead.



(a) Original RFI (b) Grayscale (c) CLAHE



(d) Gamma Adjustment (e) Min-Max Normalization (f) Median Filtering

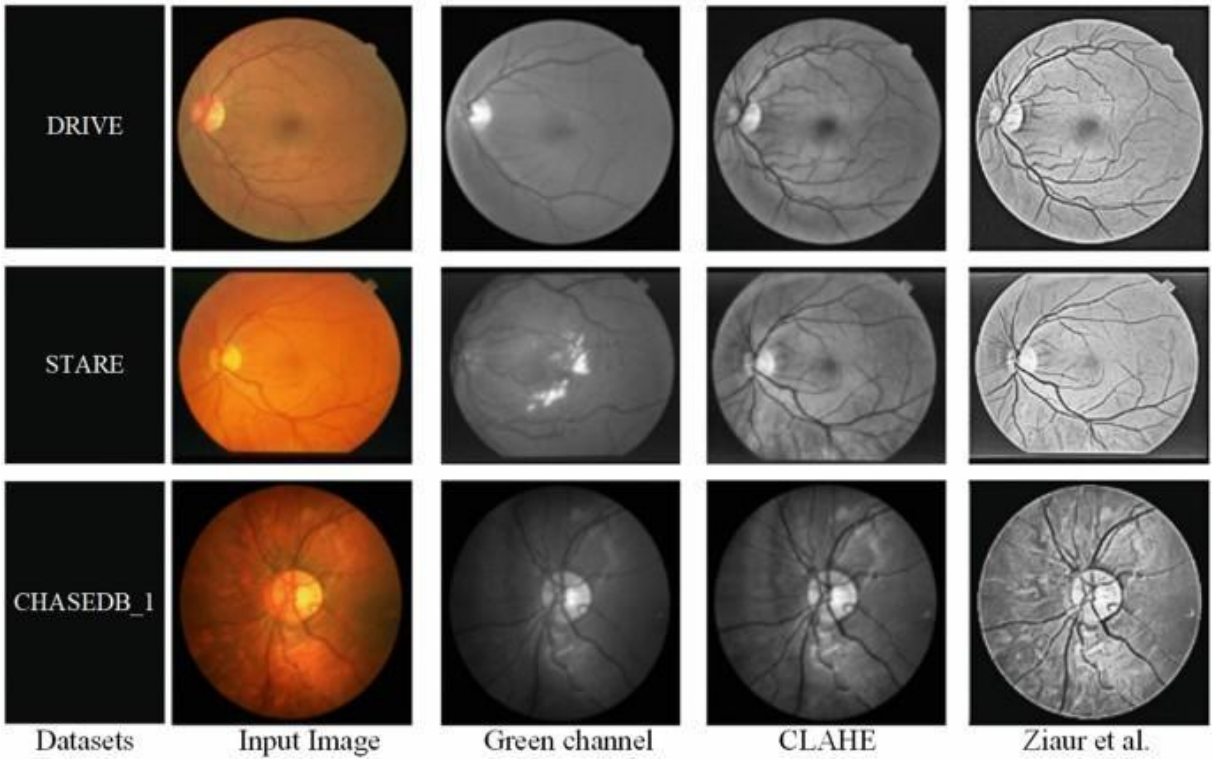


(g) Vertical flip (h) Rotation (i) Random Crop

Module B — Image preprocessing

Implement normalization, circular cropping, resizing, and contrast enhancement, including CLAHE where appropriate.

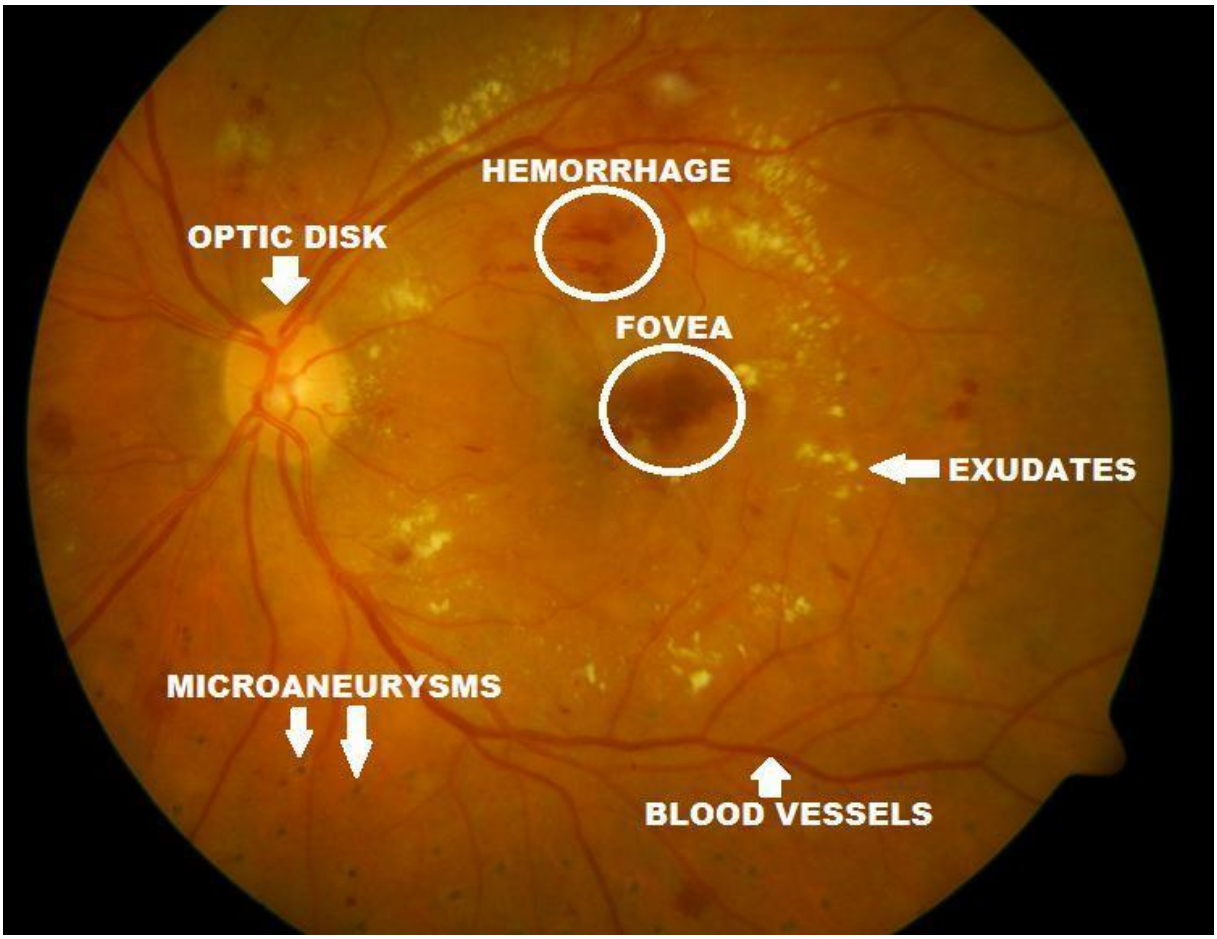
Investigate per-image processing costs, intermediate arrays, memory allocation, and opportunities for batching.



Module C — Image filters

Apply selected operations, such as Gaussian blur, median filtering, green-channel extraction, and contrast adjustment.

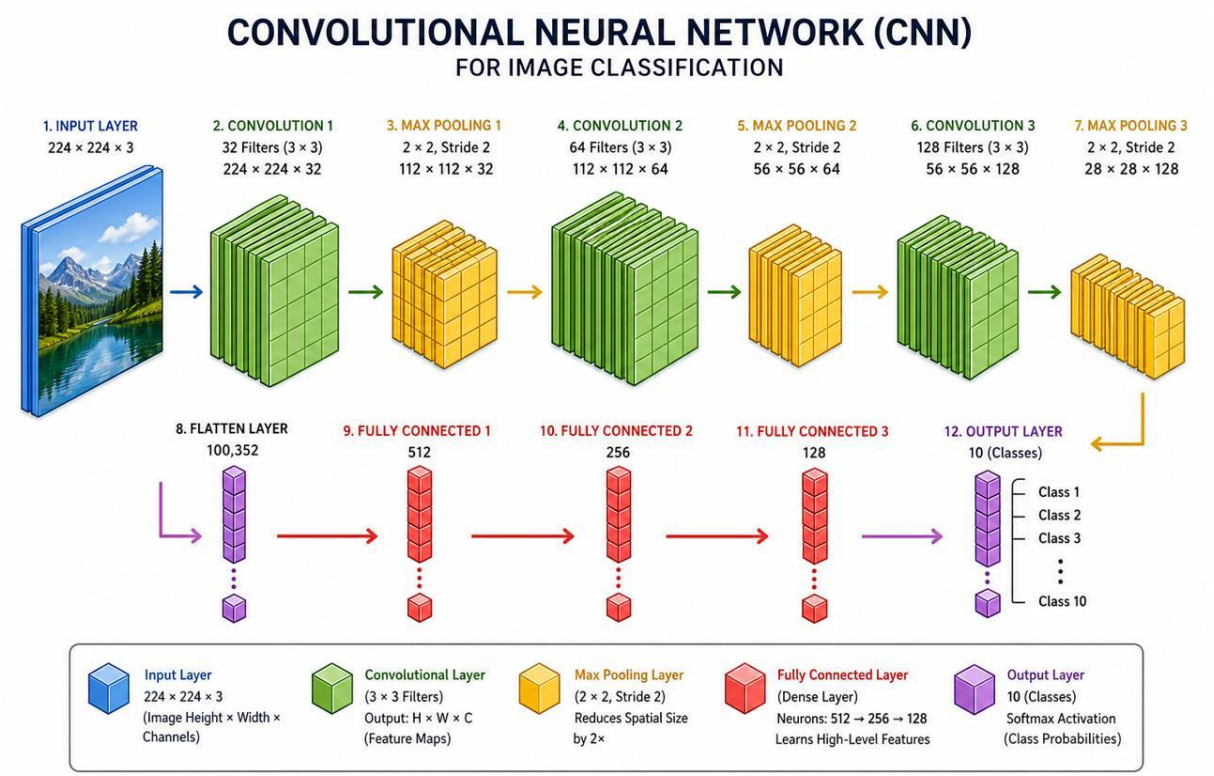
Investigate computational complexity, neighborhood operations, parallel execution, and algorithmic alternatives.



Module D — Data augmentation

Apply suitable flips, rotations, and other medically appropriate transformations to training images only.

Investigate on-the-fly versus precomputed augmentation and CPU versus GPU execution.



Module E — CNN training

Train a reference CNN using the same predefined training and validation data.

Investigate forward and backward passes, optimizer execution, batch size, mixed precision, and data-loader efficiency.

The reference pipeline must include a reproducible configuration, fixed random seeds where applicable, logging, model checkpoints, and evaluation scripts. The instructor should document the original CPU runtime and hardware, but students must independently measure and verify the baseline.

5. Project phases and weekly plan

Week 1 — Baseline execution and verification

Understand before optimizing

- Execute the complete CPU pipeline.
- Understand the source code and dependencies.
- Verify data splits, class balance, preprocessing, and model outputs.
- Record baseline classification metrics and total runtime.
- Separate one-time costs from repeated costs, such as per-epoch processing.

Deliverable: Baseline execution record and pipeline diagram.

Profiling and bottleneck identification

Measure before redesigning

- Use cProfile, line_profiler, or equivalent tools for Python execution.

- Measure image loading, preprocessing, filtering, augmentation, training, and evaluation separately.
- Use PyTorch Profiler and, when available, NVIDIA Nsight Systems to investigate CPU–GPU activity.
- Measure CPU utilization, GPU utilization, memory consumption, and data-transfer overhead where relevant.
- Rank bottlenecks by their contribution to end-to-end execution time.

Deliverable: Profiling report with timing tables, visualizations, and evidence-based optimization priorities.

Week 2 — Algorithmic and architectural redesign

Optimize the design, not just the syntax

- Identify redundant computations, unnecessary intermediate arrays, repeated disk reads, and avoidable conversions.
- Redesign preprocessing to process batches instead of individual images when appropriate.
- Compare precomputed features or preprocessing against on-demand processing, while accounting for storage and memory costs.
- Investigate algorithmic alternatives that preserve the intended image transformation.
- Write a design proposal linking each proposed optimization to a measured bottleneck.

Deliverable: Revised architecture, optimization hypotheses, and a validation plan.

GPU acceleration of image processing

Exploit parallelism where it matters

- Implement selected preprocessing and filtering operations using CuPy, Numba, or suitable GPU-enabled library functions.
- Compare library-based implementations with custom GPU implementations where justified.
- Investigate batch processing, kernel launch overhead, data transfers, and memory layout.
- Validate numerical equivalence or define acceptable tolerances.
- Profile the accelerated implementation and identify new bottlenecks.

Deliverable: CPU/GPU comparison of selected image-processing stages, including correctness tests.

Week 3 — End-to-end pipeline optimization

Reduce overhead and improve throughput

- Tune DataLoader workers, prefetching, batch size, and pinned memory where supported.

- Investigate non-blocking transfers and overlapping CPU preparation with GPU execution.
- Reduce unnecessary CPU–GPU transfers and redundant memory allocation.
- Compare precomputed preprocessing with on-the-fly GPU processing.
- Evaluate total pipeline throughput, not just isolated kernel speed.

Deliverable: Optimized input pipeline and end-to-end performance measurements.

Training acceleration and accuracy validation

Optimize training without compromising evaluation

- Move the model and training tensors to the GPU.
- Experiment with mixed precision where supported.
- Tune batch size and data-loading parameters within hardware constraints.
- Compare epoch time, training throughput, convergence, GPU memory, and validation metrics.
- Investigate whether changes in precision, batch size, or training configuration affect model quality.

Deliverable: Optimized training implementation and controlled accuracy/performance comparison.

Week 4 — Final benchmarking and defense

Demonstrate, explain, and defend the results

- Run the final baseline and optimized pipelines under controlled conditions.
- Repeat benchmarks and report variability.
- Perform an ablation study to measure the contribution of individual optimizations.
- Discuss speedup, accuracy, resource usage, limitations, and unsuccessful experiments.
- Submit the final code, technical report, and presentation.

Deliverable: Reproducible project repository, final technical report, and oral defense.

6. Required optimization strategies

Students must select strategies based on their profiling results rather than implement every technique indiscriminately.

Area	Possible strategies
Algorithm redesign	Eliminate redundant operations, reduce memory allocations, select more efficient algorithms
Image preprocessing	Batched tensor operations, GPU-compatible transformations, reduced precision where valid
Image filters	Vectorized operations, library GPU kernels, custom kernels for selected bottlenecks

Data pipeline	Caching, worker tuning, prefetching, pinned memory, asynchronous transfers
GPU memory	Reuse buffers, avoid unnecessary copies, control batch size
CNN training	GPU execution, mixed precision, batch-size tuning, optimized data loading
Whole pipeline	Overlap stages, reduce synchronization, optimize CPU–GPU boundaries

Required redesign challenge

Instead of treating custom CUDA C++ kernel development as the primary advanced challenge, students must redesign the computational pipeline to improve classification accuracy and computational efficiency simultaneously.

This redesign should include:

1. Model architecture evaluation: Compare the reference CNN with at least one justified alternative, such as a lightweight residual network or a pretrained model adapted to the dataset.
2. Input representation: Investigate whether image resolution, color representation, normalization, or contrast enhancement affects classification performance and runtime.
3. Training strategy: Explore justified changes to augmentation, regularization, learning-rate scheduling, or class-sensitive evaluation.
4. Computational efficiency: Evaluate the cost of the revised design, including preprocessing, inference, memory usage, and training.
5. Controlled comparison: Compare the original and redesigned models using the same data splits and a clearly defined experimental protocol.

The model redesign is not a license to change multiple variables without analysis. Students should isolate changes and explain whether improvements come from GPU acceleration, algorithmic optimization, or changes to the learning method.

An advanced team may also investigate model compilation, operator fusion, channels-last memory formats, or graph-based execution, depending on its hardware and software environment.

7. Reinforcement of the experimental methodology

The central rule of this project is: profile, hypothesize, optimize, validate, and re-profile.

Every major optimization must be justified by evidence.

Mandatory optimization cycle

1. Measure the baseline. Record execution time, hardware, software versions, dataset configuration, and classification metrics.
2. Identify a bottleneck. Use a profiler to show where time or memory is being spent.

3. State a hypothesis. Explain why the proposed change should improve performance and what trade-offs may occur.
4. Change one major factor at a time. Keep experiments interpretable and preserve a working reference version.
5. Validate correctness. Check output consistency, label alignment, numerical tolerances, and model evaluation.
6. Benchmark fairly. Warm up the GPU, synchronize appropriately when timing GPU work, repeat runs, and report variability.
7. Re-profile the new version. Determine whether the original bottleneck improved and whether another stage has become dominant.
8. Document the decision. Keep successful and unsuccessful experiments, including their measured costs and explanations.

Required experimental rules

- Use the same hardware for CPU/GPU comparisons whenever possible, and document any differences.
- Distinguish cold-start time, steady-state execution time, and end-to-end runtime.
- Include host-to-device and device-to-host transfers in end-to-end timings.
- Avoid timing asynchronous GPU operations without proper synchronization.
- Run each major benchmark multiple times and report the mean and either standard deviation or a suitable variability measure.
- Use the same dataset splits, evaluation protocol, and comparable training budgets when comparing models.
- Report the exact software versions, hardware specifications, batch sizes, image dimensions, and relevant configuration settings.
- Separate speedups attributable to GPU execution from speedups attributable to algorithm or model redesign.

8. Final deliverable: structure and submission guidelines

Each team must submit a self-contained repository and a technical report. The report must be understandable to another engineering student who wishes to reproduce the experiments.

A. Source code and repository

Recommended structure:

```
gpu-dr-project/
├── README.md
├── requirements.txt
├── configs/
│   ├── baseline.yaml
│   └── optimized.yaml
├── src/
│   ├── data_loader.py
│   ├── preprocess_cpu.py
│   ├── filters_cpu.py
│   ├── model.py
│   ├── train_cpu.py
│   ├── preprocess_gpu.py
```

```
|   ├── filters_gpu.py
|   ├── train_gpu.py
|   └── evaluate.py
├── benchmarks/
|   ├── profile_baseline.py
|   ├── benchmark_stages.py
|   └── benchmark_end_to_end.py
├── tests/
|   ├── test_preprocessing.py
|   └── test_cpu_gpu_equivalence.py
├── results/
|   ├── timing.csv
|   ├── metrics.json
|   └── figures/
├── report/
|   └── final_report.pdf
└── environment/
    └── hardware_software.md
```

The directory names may be adapted, but the separation between reference implementation, optimized implementation, tests, benchmarking, and results must remain clear.

The dataset itself does not need to be included in the submitted archive if it is distributed separately. Provide instructions for configuring its location.

The README.md must explain:

- Project objective and team members.
- Environment setup and dependencies.
- Dataset directory structure and expected labels.
- How to run the CPU baseline.
- How to run the optimized implementation.
- How to reproduce profiling and benchmarking.
- How to reproduce model evaluation.
- Known compatibility issues and hardware requirements.

B. Technical report

Recommended length: 10–15 pages, excluding appendices and references.

Required report outline

1. Abstract and objectives
   Summarize the problem, methods, main measured results, and most important conclusion.
2. Problem and pipeline description
   Explain the dataset, classification task, CPU reference implementation, and computational stages.
3. Experimental environment
   Specify CPU, GPU, RAM, operating system, framework versions, image resolution, data splits, and training configuration.
4. Baseline profiling
   Present the profiling methodology, time distribution, resource utilization, and identified bottlenecks.

5. Redesign and optimization decisions
   Explain each proposed change, its motivation, implementation, expected effect, and potential trade-offs.
6. Benchmarking methodology
   Describe warm-up, synchronization, repetition count, timing boundaries, and how variability was measured.
7. Results and comparisons
   Present stage-level timings, end-to-end speedup, training throughput, memory consumption, and classification metrics.
8. Discussion and critical analysis
   Explain why the optimizations succeeded or failed, identify bottlenecks that remain, and distinguish algorithmic from hardware-related gains.
9. Limits and threats to validity
   Discuss dataset size, generalization, hardware dependence, numerical differences, measurement uncertainty, and reproducibility.
10. Conclusion and future work
    Summarize the contribution, strongest evidence, lessons learned, and next steps.

C. Mandatory results tables and figures

Students must include the following evidence.

Table 1 — Hardware and software configuration

Record the CPU, GPU, RAM, operating system, Python version, framework versions, and relevant configuration settings.

Table 2 — Stage-level performance

Pipeline stage	CPU baseline (s)	Optimized (s)	Speedup
Image loading			
Preprocessing			
Filtering			
Augmentation			
Training per epoch			
Evaluation			
Total pipeline			

Report non-applicable or combined stages explicitly instead of presenting invented measurements. If training is measured over multiple epochs, report both per-epoch and total training time.

Table 3 — Classification quality

Metric	Baseline model	Final model
Accuracy		
Macro-precision		
Macro-recall		

Macro-F1		
Confusion matrix	Included	Included

Because the classes are balanced, accuracy is informative, but macro-F1 and per-class results are still important. If clinically meaningful sensitivity or specificity is investigated, define exactly how each metric is calculated for the multiclass problem.

Figure 1 — Baseline runtime breakdown. Show the contribution of each stage to total runtime.

Figure 2 — Speedup comparison. Compare stage-level speedups and end-to-end speedup.

Figure 3 — Accuracy versus execution time. Show the performance–quality trade-off between tested configurations.

Figure 4 — Ablation study. Show the incremental contribution of individual optimizations or clearly defined combinations.

Figure 5 — Resource utilization. Include relevant CPU utilization, GPU utilization, and memory measurements where tools permit.

D. Required quantitative analysis

For every measured stage, calculate speedup as:

$$S = \frac{T_{\text{baseline}}}{T_{\text{optimized}}}$$

For example, if a stage takes 20 seconds in the baseline and 5 seconds after optimization, its speedup is \(4\times\).

Report the end-to-end speedup separately from individual kernel or stage speedups. A fast kernel does not necessarily produce a fast application.

Students should also discuss Amdahl's law:

$$S_{\text{total}} = \frac{1}{(1-p) + \frac{p}{s}}$$

where \(p\) is the fraction of baseline execution time affected by the optimization and \(s\) is the speedup of that part.

This helps explain why accelerating a small fraction of total runtime may have little impact on overall performance.

E. Final oral defense

Each team should deliver a 15–20-minute presentation followed by questions.

The presentation must answer:

- What was the principal bottleneck, and what evidence established it?
- Which redesign or GPU optimization delivered the greatest end-to-end improvement?

- Which optimization had less benefit than expected, and why?
- Did classification quality change?
- What are the remaining bottlenecks?
- Which results can be reproduced on another machine, and which depend on the hardware?

Students must be able to explain their own code and defend the validity of their measurements.

9. Discussion of results: mandatory critical analysis

The report must go beyond presenting tables and claiming that the GPU is faster.

Students should explicitly discuss the following questions.

Performance

- Why did some stages benefit more from GPU execution than others?
- Did data transfer, memory allocation, synchronization, or kernel launch overhead reduce the gains?
- Did the bottleneck shift after the first optimization?
- Was the improvement in the isolated operation reflected in the end-to-end pipeline?

Classification quality

- Did preprocessing changes preserve important retinal structures?
- Did the redesigned model improve macro-F1 or per-class recall?
- Were changes in accuracy caused by architecture, preprocessing, precision, augmentation, or random variation?
- Was the final evaluation performed on data that was not used for model selection?

Engineering trade-offs

- Did the solution require more GPU memory or disk storage?
- Did an optimization make the code more complicated or less portable?
- Would the same strategy be beneficial on a different GPU or CPU?
- Was the added complexity justified by the measured improvement?

A successful project is not necessarily the one with the largest speedup. It is the one that explains the observed results rigorously and demonstrates that its conclusions are supported by reproducible experiments.

10. Limits and constraints

The project must acknowledge the limits of the experimental setup and the conclusions that can be drawn.

Dataset limitations

A dataset of 3,000 images is appropriate for an educational project but is not sufficient, by itself, to establish clinical readiness.

- Balanced classes simplify experimentation but may not reflect real-world DR prevalence.
- Results may depend on image quality, camera characteristics, acquisition conditions, and patient populations.
- Data leakage, especially between images from the same patient, can inflate reported performance.

Model and clinical limitations

- Classification performance on the provided dataset does not establish clinical safety or generalization to other hospitals.
- The project is an educational benchmark, not a diagnostic medical device.
- Improvements in accuracy do not automatically imply improvements in clinical utility.
- Any claim of improved classification must be supported by controlled experiments and appropriate evaluation metrics.

GPU and performance limitations

- GPU acceleration depends on the hardware, workload size, framework, and memory constraints.
- Small operations may run faster on a CPU because GPU transfers and launch overhead dominate.
- Some operations may have limited parallelism or depend on sequential processing.
- Different GPUs, driver versions, libraries, and precision modes can produce different results.
- Training time and inference time are distinct metrics and must not be conflated.

Methodological limitations

- Different training seeds may produce different model results.
- Comparing different architectures, image resolutions, and training budgets simultaneously makes causal attribution difficult.
- Precomputing data can improve runtime while increasing storage and preprocessing costs.
- Profiling itself may introduce overhead; measurements must use appropriate tools and clearly defined timing boundaries.
- A speedup measured on one machine cannot automatically be generalized to another.

Scope constraints

Within seven weeks, students are not expected to implement every possible optimization or produce a clinically validated model. They must instead select a manageable set of high-value improvements and demonstrate a rigorous optimization process.

11. Evaluation rubric

Criterion	Weight
Baseline execution and understanding	10%
Profiling methodology and bottleneck identification	20%
Algorithmic redesign and GPU implementation	20%
Experimental rigor and reproducibility	15%
Performance analysis and discussion	15%
Classification evaluation and limitations	10%
Code quality, report, and oral defense	10%
Total	100%

Assessment should reward sound reasoning, correct measurements, and reproducibility, not just raw speedup. Students who identify a bottleneck correctly and demonstrate why a proposed optimization is ineffective should receive credit for the quality of their analysis.

12. Optional advanced challenges

For teams that complete the core objectives early:

- Compare multiple GPU architectures or GPU generations, if available.
- Evaluate model compilation and operator fusion.
- Compare different input resolutions and quantify the accuracy–latency trade-off.
- Investigate inference optimization separately from training optimization.
- Evaluate the impact of batch size on throughput, latency, and GPU memory.
- Develop a simple performance model to predict when CPU execution is preferable to GPU execution.
- Compare the energy consumption of CPU and GPU execution if reliable measurement tools are available.

Conclusion

This project is designed to teach students a fundamental principle of high-performance computing: performance optimization begins with understanding the workload, not with choosing a technology.

By working with a real retinal image classification pipeline, students will learn to connect algorithm design, computer architecture, GPU programming, machine learning, and experimental science. They will discover that accelerating one operation is not the same as accelerating an application, that faster execution must be measured rather than assumed, and that computational gains are meaningful only when correctness and model quality are preserved.

The final objective is for every team to deliver more than a GPU-enabled version of the original code. Each team must produce a measurably improved, reproducible, well-documented implementation, supported by profiling evidence, controlled experiments, critical discussion, and a clear account of its limitations.

Ultimately, the project should prepare future engineers to approach performance problems systematically: measure first, redesign intelligently, optimize selectively, validate rigorously,

and defend conclusions with evidence.